COMS W4156 Advanced Software Engineering (ASE)

October 17, 2023

Upcoming Assignments

- ➔ First Iteration due this Thursday, October 19, by 11:59pm
- ➔ First Iteration Demo due next Thursday, October 26, by 11:59pm

First Iteration vs. First Iteration Demo

By the time you submit your First Iteration assignment, all the pieces should be in place in your github repo (possibly preliminary versions of some of the pieces)

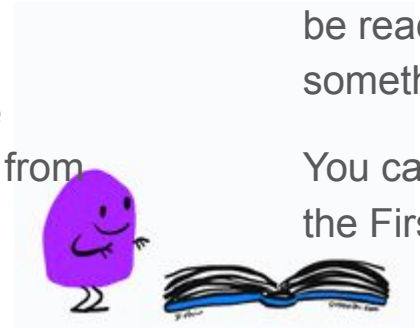
By the time you submit your First Iteration Demo assignment, your service should work, you should be able to show that it works, and your Mentor should be able to "drive"

First Iteration all the pieces in place

Your README should document your API (or link to your external API docs). It should also contain instructions (that work!) for how to build, run and test your service

Your repo should contain system-level tests exercising your API entry points "over the network" - this can be localhost but should not be in-process or otherwise assume same machine

Some of the API tests should involve persistent data and pretend to come from different clients



Your repo should contain unit tests for most major methods, with setup/teardown and mocks when applicable, that run "push button" from a unit testing tool

Your repo should contain some style-checker reports and most of its complaints should have been fixed. Your code should be commented, use mnemonic identifiers and be readable. Commit messages should say something meaningful about what changed

You can continue updating all of these until the First Iteration Demo and beyond

Agenda

1. First Iteration vs. First Iteration Demo
2. recording beliefs for debugging
3. recording who did what, where and when for security and compliance

You Believe Your Code Is Correct and Complete

Localizing a bug is a process of checking beliefs about the program

- ➔ Developer might believe that a function was called with particular parameters
- ➔ Developer might believe that a certain variable has a certain value at a certain place in the code
- ➔ Developer might believe that an API or library call returned a specific value
- ➔ Developer might believe that a particular execution path is taken through conditionals

You know what your beliefs are while writing the function, so add these beliefs to each function while writing it using logging

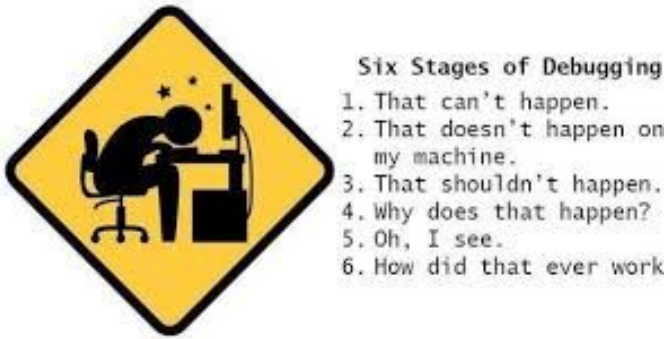
Comments help but logs are better, because they can be checked

Code Beliefs

```
int foo (int a,  int b) {
        int c;
        ...
        code does something with a and b
        …
        c = external_call(a,b);
        ...
        code does something with a, b and c
        …
        return c;
}
```

```
int foo (int a,  int b) {
        int c;
                [ believe a>0 and b>0 ]
        ...
                [ believe a>0 and b>0 ]
        c = external_call(a,b);
                [ believe a>0 and b>0, believe c != 0 ]
        …
                [ believe a>0 and b>0, believe c != 0 ]
        return c;
}
```

Checking Beliefs During Debugging



If you execute the function in an interactive debugger *soon after writing it when you still remember all your beliefs*, you can manually check that your beliefs are correct for a small number of test cases. Set breakpoint at beginning of function and examine the values of the parameters, then single-step through code while examining values that could have changed

More commonly interactive debugging happens later, after tests fail, and the developer (or your future self) may not know what the beliefs were unless something present in the code tells them

They cannot know whether those beliefs were correct during previous executions unless saved execution logs tell them

Code Beliefs

```
int foo (int a,  int b) {
        int c;
        ...
        code does something with a and b
        …
        c = external_call(a,b);
        ...
        code does something with a, b and c
        …
        return c;
}
```


```
int foo (int a,  int b) {
        int c;
                [ breakpoint to check a>0 and b>0 ]
        ...
                [ breakpoint to check a>0 and b>0 ]
        c = external_call(a,b);
                [ breakpoint to check a>0, b>0, c != 0 ]
        …
                [ breakpoint to check a>0, b>0, c != 0 ]
        return c;
}
```

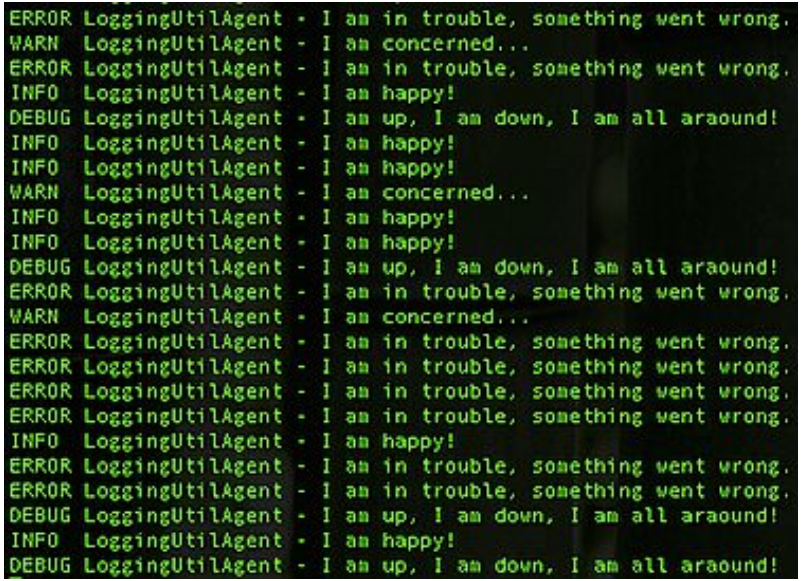
Logging Records Belief Checks Automatically

Use logging package instead of print statements!

Log entries are just messages that describe events or state during execution

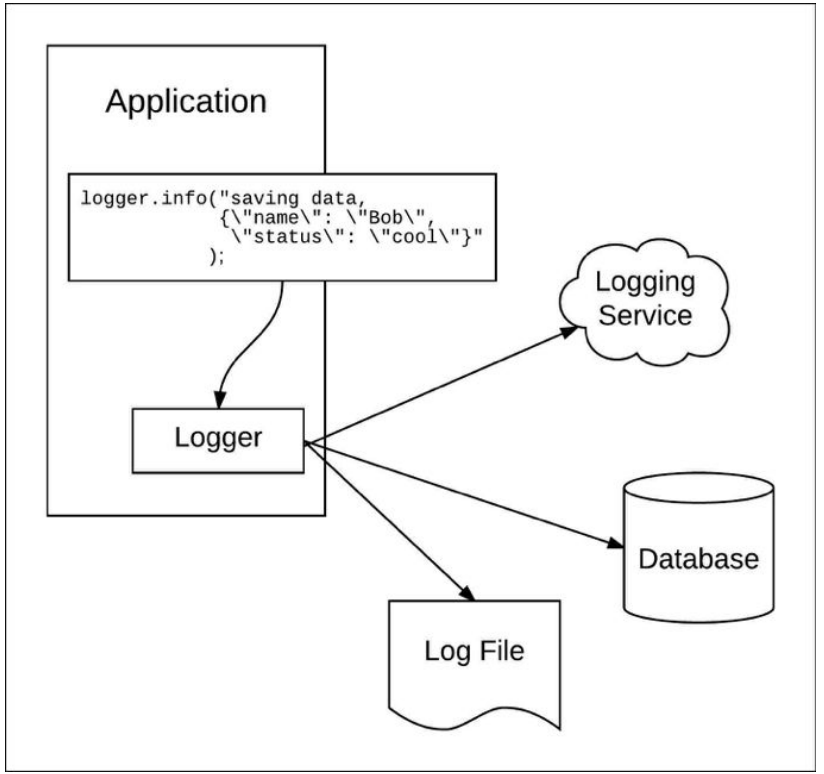
Severity levels might be configured so all levels are logged during development with only the most severe levels logged in production - the other logging statements are not necessarily removed, they just don’t do anything

Turning off some log statements during production reduces log size and logging overhead.
❓Why is there overhead ❓

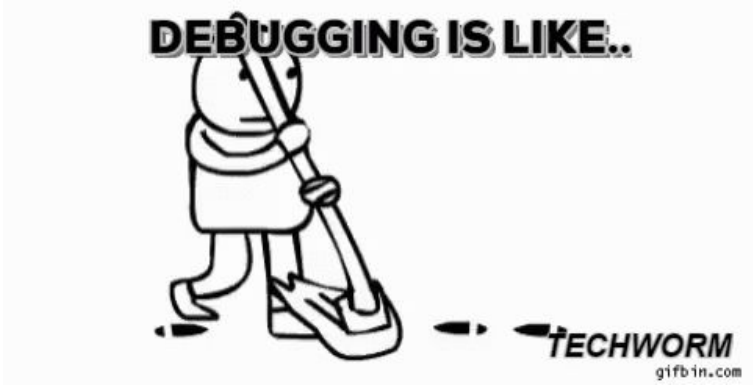


Logging Beliefs

```
int foo (int a,  int b) {
        int c;
        log(“beginning of foo”,a,b);
        ...
        log(“middle of foo”,a,b);
        c = external_call(a,b);
        log(“after foo calls external_call”,a,b,c);
        …
        log(“end of foo”, a,b,c);
        return c;
}
```



Why Not Use Print Statements?



What is wrong with print statement debugging? It's what everyone learned first, it "worked", why give it up?

Introducing print statements *changes* the code - modern languages are designed to prevent semantic changes due to built-in (or standard library) debugging statements, but cannot help if you introduce new code that cannot be distinguished from program code

This problem is most significant with C/C++ and other languages with unmanaged memory, but can occur in nearly any language

What Are Severity Levels?

By setting the log level appropriately, developers can
filter out irrelevant information and focus on what's important

- **Debug:** Show diagnostic information about the system.
- **Info:** Information about major events in a systems's regular operation, such as a successful system startup or shutdown.
- **Warning:** Runtime issues that do not result in an error, but may show other problems that could eventually result in an error, such as system instability or connectivity issues.
- **Error:** Runtime issues that violate the requirements of a system, such as unhandled input or outputs to the process.
- **Critical:** Events that violate the system's requirements such that they will generally result in the shutdown of the system.

Setting the log level too low can result in missing important information; putting it too high can generate excessive noise, making it harder to find essential logs and slowing down the system

Generate Diagnostic Information

When debug logging, you will want to record *detailed* diagnostic information to make it easier to understand what your program is doing:

- Timestamps of events
- (Partial) ordering of events across different threads/processes
- Resource usage - what, when, how much
- Entity IDs and Session IDs for communicating entities
- Data received from/sent to those entities
- Values of key variables

❓What is the difference between entity IDs and session IDs, and why does it matter❓

Protect Sensitive Log Data

It is essential to protect sensitive information when logging in production environments or when production data is used in testing

For example, debug logs accessible to ordinary developers should not record personal identifying information (PII)

When PII data must be logged, e.g., audit logs, the data should be secured through log file permissions and securely shipped to an external log repository

Benefits of Debug Logging

Assistance in identifying root cause: When an error or unexpected behavior occurs, developers can examine the log files to understand what happened and identify the root cause of the issue. Debug logging can provide a wealth of information about the system's state at the time of the error, including the values of variables and other in-memory data, the state of the environment, and the sequence of events that led to the error.

Insight into how to fix an error: Just as debug logging information helps isolate the root cause of an error, that same information provides insights into how developers can implement a fix, e.g., what code was running when the bug manifested (the log diverged from expected behavior)?

Insight into how to reproduce an error: When edge cases are hit during runtime, the information gleaned from debug logging can make bug reproduction easier

Early identification of potential performance bugs: By logging data during runtime, such as time of request and time of corresponding response, developers can better understand a program's performance and identify potential bottlenecks or areas for optimization

Some Logging Packages

[loggly ultimate guide](#) covers many languages and platforms

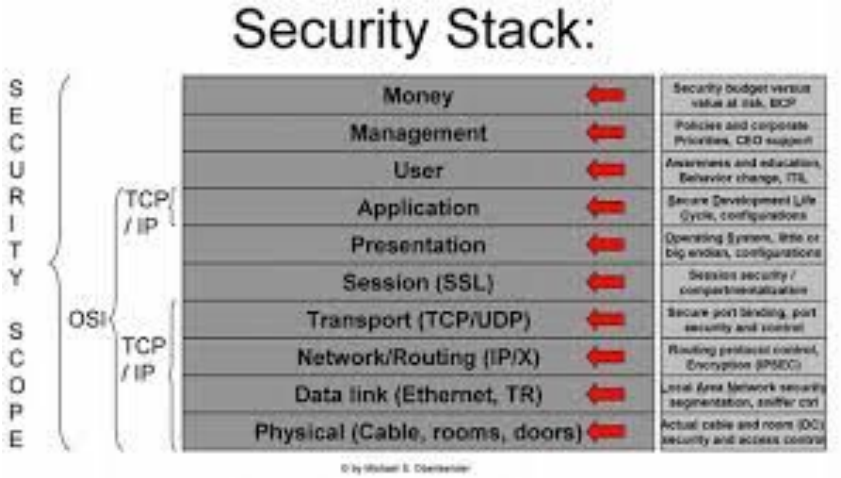
[logging for C++](#)

[logging for Java](#)

[logging for Node](#)

Agenda

1. First Iteration vs. First Iteration Demo
2. recording beliefs for debugging
3. recording who did what, where and when for security and compliance



Audit Logs

Primarily concerned with documenting who did what when during production execution. Considered part of the functionality of the system, so logging itself needs to be tested and debugged not just treated as an auxiliary to other testing and debugging

Industry standards and government regulations — such as the Payment Card Industry Data Security Standard (PCI DSS), the Health Insurance Portability and Accountability Act (HIPAA), the Sarbanes-Oxley (SOX) Act and the Gramm-Leach-Bliley Act— require companies to create and keep audit logs for compliance, and may require covering specific categories of events. For example, PCI DSS requires monitoring access to cardholder data

Reconstruct timeline of system outage or security breach, provide legal evidence

What Kinds of Events Are Logged

Creating/deleting accounts, registration keys, etc.

Login/logout and connect/disconnect successes and failures

Successful and denied use of privileges (e.g., sudo)

Successful and denied access to sensitive data - create, query, view, modify, copy, delete, etc.

Platform level changes such as editing or replacing configuration files, creating new VM instance, installing software, updating library version, etc.

Record Higher Level Data Than Debug Logs

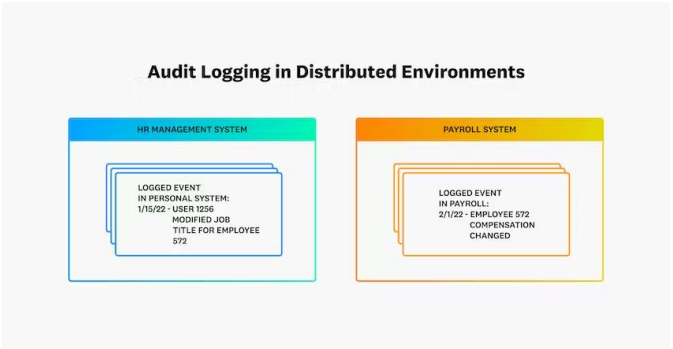
- Event name as identified in the system
- Easy-to-understand description of the event
- NTP-synced event timestamp
- Actor or service that created or caused the event (user ID, client or API ID)
- Application, service, system, device, resource that was impacted (IP address, device ID, etc.)
- Source from where the actor or service originated (country, host name, IP address, device ID)
- Custom tags such as severity level of the event

Audit Log Retention

Whereas regular system logs are designed to help developers troubleshoot errors, audit logs help organizations document a historical record of activity for compliance purposes and other business policy enforcement

Organizations must meet long-term retention policies (e.g., banks must keep deposit account records for five years)

Audit logs must be *tamper-proof and immutable* so no user or software can alter audit trails (audit trail = time series of audit logs) - no log rotation, no other reuse of log files, no log truncation, no deleting old log files to clear up space



Audit Log Management and Analysis Tools

Most audit-related software costs $$$s,
some doesn't:

graylog open

syslog-ng open source edition

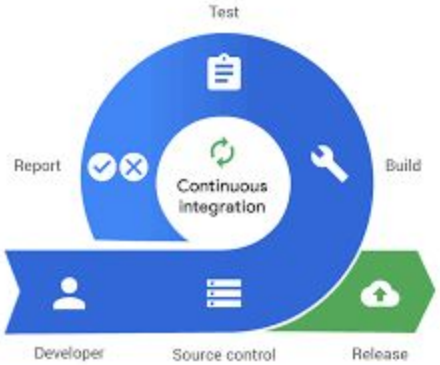
AlienVault OSSIM

7 Best Open Source SIEM Systems
to Improve Your Cyber Security

SIEM = security information and event management

Thursday

continuous integration with demo



Upcoming Assignments

- ➔ First Iteration due this Thursday, October 19, by 11:59pm
- ➔ First Iteration Demo due next Thursday, October 26, by 11:59pm

First Iteration vs. First Iteration Demo

By the time you submit your First Iteration assignment, all the pieces should be in place in your github repo (possibly preliminary versions of some of the pieces)

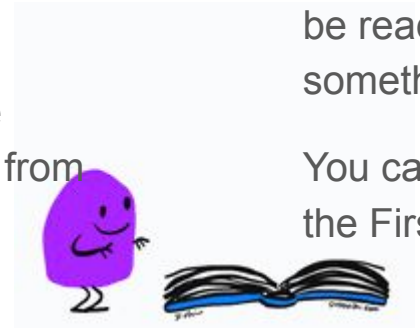
By the time you submit your First Iteration Demo assignment, your service should work, you should be able to show that it works, and your Mentor should be able to "drive"

First Iteration all the pieces in place

Your README should document your API (or link to your external API docs). It should also contain instructions (that work!) for how to build, run and test your service

Your repo should contain system-level tests exercising your API entry points "over the network" - this can be localhost but should not be in-process or otherwise assume same machine

Some of the API tests should involve persistent data and pretend to come from different clients



Your repo should contain unit tests for most major methods, with setup/teardown and mocks when applicable, that run "push button" from a unit testing tool

Your repo should contain some style-checker reports and most of its complaints should have been fixed. Your code should be commented, use mnemonic identifiers and be readable. Commit messages should say something meaningful about what changed

You can continue updating all of these until the First Iteration Demo and beyond

Ask Me Anything